

**TEMPLATE
PROJECT WORK**

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (<i>pari a circa Minimo 12 – Massimo 20 pagine</i>)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Socci Arianna
Numero di matricola	312402195
Tema n. (Indicare il numero del tema scelto):	1
Titolo del tema (Indicare il titolo del tema scelto):	La digitalizzazione dell'impresa
Traccia del PW n. (Indicare il numero della traccia scelta):	10
Titolo della traccia (Indicare il titolo della traccia scelta):	Sviluppo di una dashboard in Java per l'analisi delle prestazioni aziendali nel settore primario
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Real-time Decision Support System per l'Agritech

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Le conoscenze e le abilità acquisite durante il percorso di studi sono risultate fondamentali per la realizzazione dell'elaborato, in particolare per la progettazione e lo sviluppo dell'applicativo. L'insegnamento di "Ingegneria del software" ha fornito gli strumenti metodologici per seguire un processo strutturato di analisi dei requisiti, progettazione architettuale, sviluppo e test.

In particolare, le competenze relative al pattern MVC (Model–View–Controller), un'architettura che suddivide l'applicazione in tre componenti distinte: Model, che rappresenta i dati e la logica di dominio; View, responsabile della presentazione; e Controller, che gestisce le interazioni dell'utente e coordina il flusso tra modello e vista, hanno permesso di mantenere una chiara separazione delle responsabilità, riducendo le dipendenze tra le parti del sistema e favorendo la manutenibilità del codice. A queste si aggiungono le responsabilità relative alla gestione dei repository e alla scrittura di una documentazione presente sia nel codice che sul progetto. Nel loro insieme, queste parti hanno permesso di organizzare l'intero progetto in fasi coerenti, garantendo la modularità.

La modellazione UML, la definizione dei casi d'uso e la descrizione dei flussi di interazione tra i componenti hanno inoltre contribuito a costruire una rappresentazione chiara e sistematica del funzionamento dell'applicazione, così che potessi avere un'idea concreta di quello che avrei voluto implementare prima di raggiungere la fase di sviluppo effettivo.

Il corso di "Tecnologie Web" ha avuto un ruolo centrale nella realizzazione del frontend, fornendo conoscenze approfondite su HTML, CSS, JavaScript, comunicazione client-server e interfacce dinamiche. Queste competenze hanno permesso di sviluppare una dashboard interattiva e responsiva,

con funzionalità come la visualizzazione grafica dei dati, filtri per area geografica e tipologia di metrica, e l'integrazione di aggiornamenti in tempo reale tramite WebSocket, migliorando l'esperienza utente e la fruibilità delle informazioni. La comprensione del DOM (Document Object Model) ha permesso di implementare funzionalità interattive, integrando in modo armonioso la componente visuale con il backend scritto in Java. Infine, l'approfondimento sugli standard del web, sui protocolli di comunicazione come HTTP e HTTPS, sulla gestione delle richieste asincrone e sull'utilizzo di template dinamici ha reso possibile un'integrazione fluida tra frontend e backend.

L'insegnamento di "Programmazione distribuita e cloud computing" è stato particolarmente utile per comprendere e implementare la logica di aggiornamento in tempo reale, la gestione delle connessioni persistenti e l'architettura modulare del sistema. I principi appresi relativi ai sistemi distribuiti, alla scalabilità dei servizi e alla gestione dei dati in contesti in-memory hanno consentito di progettare un'applicazione capace di simulare scenari realistici, gestire aggiornamenti simultanei e garantire la coerenza tra dati ambientali e produttivi, mantenendo una struttura flessibile e pronta per futuri ampliamenti con dataset reali o deployment distribuiti. Inoltre, i concetti di virtualizzazione, containerizzazione e architetture cloud illustrati nel corso hanno offerto gli strumenti teorici per valutare opzioni di distribuzione dell'applicativo in ambienti scalabili, ponendo le basi per un futuro passaggio verso infrastrutture distribuite, microservizi o soluzioni serverless (un'approfondimento di questo punto verrà trattato nella sezione: "Valutazione dei risultati").

A questi tre pilastri formativi si aggiunge un ulteriore livello di integrazione derivante dall'applicazione pratica dei contenuti appresi: le metodologie di analisi e progettazione studiate in Ingegneria del Software si sono combinate con le tecnologie presentate nel corso di Tecnologie Web, mentre le nozioni su sistemi distribuiti e cloud hanno permesso di considerare l'applicativo non come un semplice prototipo locale, ma come un sistema potenzialmente scalabile, capace di operare in scenari con un elevato volume di dati e molteplici utenti. Il progetto, nonostante la sua semplicità, risulta per questo fondato su solide basi teoriche e su scelte progettuali consapevoli.

In aggiunta, è opportuno richiamare alcuni insegnamenti seguiti presso l'Università di Pisa, università dalla quale ho effettuato il trasferimento, i quali hanno fornito competenze preliminari rilevanti per lo sviluppo del presente lavoro. Il corso di Programmazione 1 ha introdotto i fondamenti della programmazione a oggetti, rivelatisi essenziali per impostare un'architettura basata su classi modulari e coerenti. La padronanza dei concetti di incapsulamento, ereditarietà e polimorfismo ha infatti consentito di definire modelli di dominio robusti e di garantire la manutenibilità del codice lungo l'intero ciclo di sviluppo.

Un ulteriore riferimento merita l'insegnamento di Reti degli Elaboratori, anch'esso svolto presso l'Università di Pisa, che ha offerto strumenti concettuali e pratici utili alla progettazione e all'implementazione della comunicazione tramite WebSocket. Questi contenuti hanno integrato efficacemente le competenze maturate nel percorso attuale, contribuendo alla definizione di un impianto progettuale più solido e consapevole.

Grazie a questi corsi è stato possibile comprendere i principi di comunicazione bidirezionale tra client e server, gestire connessioni persistenti e implementare una dashboard chiara ed esaustiva con codice scalabile e manutenibile, elementi chiave per la funzionalità di aggiornamento continuo della dashboard e la trasmissione immediata dei dati simulati agli utenti.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato

(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Analisi dei requisiti e definizione dell'obiettivo (3gg).

La fase iniziale ha riguardato la definizione dell'ambito del progetto e delle entità chiave, quali Crop, Region, EnvironmentalData e ProductionData. In questa fase sono stati formalizzati anche due

stream di dati separati, rispettivamente Ambientale e Produttivo, con l'obiettivo di garantire una distinzione chiara tra le informazioni relative alle condizioni ambientali e quelle relative ai risultati di produzione. La definizione delle entità ha permesso di strutturare fin da subito il modello di dominio in maniera coerente e facilmente scalabile, questo porterà un vantaggio alle possibili implementazioni future.

Design architetturale (2gg).

Il progetto è stato progettato seguendo il pattern MVC (Model-View-Controller), suddividendo chiaramente le responsabilità tra Controller, Service, Repository e Model. La scelta di un database in-memory (H2) mi ha permesso di dedicare maggiore attenzione allo sviluppo applicativo, eliminando il fattore di complessità legato alla persistenza e la necessità di gestire database esterni. Durante la progettazione è stata valutata l'adozione di due possibili strategie di trasmissione dati al front-end: un sistema basato su polling, ovvero con il client che chiede periodicamente al server se ci sono nuovi dati, ed uno basato su push, cioè con il server che invia attivamente i dati non appena disponibili. Per motivi di semplicità, scalabilità e facilità di sottoscrizione da parte della dashboard, ho deciso di implementare un sistema STOMP over WebSocket, una connessione persistente aperta tra client e server, permettendo aggiornamenti in tempo reale senza sovraccaricare l'interfaccia utente. Il modello basato su WebSocket risulta infatti significativamente più performante del tradizionale polling nelle applicazioni reali, poiché elimina la necessità di richieste ripetute e riduce in modo sostanziale il carico computazionale e di rete, che imporrebbe altrimenti al server la gestione di un elevato numero di connessioni brevi, ridondanti e spesso prive di nuovi dati. Ciò apporta un miglioramento alla latenza percepita e si traduce nella possibilità di supportare un'utenza più ampia, poiché il server mantiene un minor numero di operazioni superflue e può dedicare la capacità elaborativa alla gestione effettiva degli eventi applicativi.

Ricerca, sviluppo software e test (20gg).

Lo sviluppo del software è stato condotto in parallelo con la ricerca necessaria alla definizione di una base dati affidabile. La consultazione di varie fonti online e la successiva estrapolazione dati per mezzo dell'intelligenza artificiale, ha permesso di stabilire criteri coerenti per collegare tra loro le variabili e definire un modello informativo che rispecchiasse fenomeni realistici. Lo sviluppo è partito dalla classe responsabile della generazione della base di dati ambientali per la simulazione ed ho riscontrato da subito una difficoltà poiché non bastava generare numeri casuali, ma era necessario garantire che i dati avessero distribuzioni plausibili e rispettassero vincoli di dominio per creare una base utile e credibile ai fini di simulazione e sviluppo dell'applicazione (es. la correlazione tra umidità del suolo e pioggia/vento, approfondiremo questo punto nel paragrafo "Descrizione dei principali aspetti progettuali", nella sottosezione "Resoconto del processo di sviluppo del codice"). L'implementazione del simulatore ha previsto quindi la creazione di modelli stagionali, come funzioni sinusoidali per la temperatura, la definizione di dinamiche correlate di umidità del terreno basate su accumulo ed evaporazione e l'introduzione di rumore gaussiano per garantire variabilità nei dati.

Il livello web, realizzato con Thymeleaf e JavaScript, sottoscrive le destinazioni tramite WebSocket e aggiorna dinamicamente i grafici della dashboard. Per garantire la correttezza dei dati e l'integrità dello stato, la produzione dei dati di simulazione inizia solo dopo che il contesto dell'applicazione è completamente stabilizzato. Durante questa fase ho riscontrato un'altra problematica significativa legata alla quantità di dati generati durante le prime simulazioni: emissioni troppo frequenti saturavano l'interfaccia, causando difficoltà di lettura dei grafici. Inoltre, ho riscontrato un risultato simile anche su generazioni con periodi di tempo troppo estesi dati in input, come ad esempio una generazione di dati ogni sei ore per un anno. La soluzione adottata è stata l'aggregazione dei dati ambientali, limitando ad un massimo di cento punti la visualizzazione dei grafici sul frontend. Per

quanto riguarda i dati di produzione, invece, l'aggregazione non risultava efficace poiché i dati non si rivelavano abbastanza esplicativi. Per garantire chiarezza e comprensibilità, ho quindi preferito utilizzare un grafico a torta invece di mantenere l'istogramma usato per i dati ambientali. Ciò ha migliorato di molto la leggibilità e l'esperienza utente.

Documentazione e revisione finale (3gg).

L'ultima fase del lavoro ha riguardato la redazione del file [README.md](#), per la quale ho impiegato strumenti di intelligenza artificiale, ormai ampiamente diffusi nel settore dello sviluppo software. In particolare, l'utilizzo di GitHub Copilot ha supportato la stesura di un documento introduttivo completo e coerente, facilitando la presentazione dell'applicazione e delle sue funzionalità principali. Successivamente, ho proceduto alla revisione della documentazione e delle dipendenze, verificando la corretta costruzione del progetto e validando i comandi necessari all'avvio del sistema e al test del suo funzionamento.

Infine, ho pubblicato il codice sul mio repository pubblico GitHub https://github.com/AriannaSocci/PW_1_10.git, rendendo il progetto accessibile e consultabile da parte di terzi attraverso un semplice comando Git: "git clone https://github.com/AriannaSocci/PW_1_10.git".

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

L'orientamento applicativo si è basato sulla documentazione ufficiale di Spring (<https://docs.spring.io/spring-framework/reference/index.html>), sulla mia esperienza lavorativa e sulle nozioni imparate nei corsi già precedentemente citati (es. Ingegneria del software e Tecnologie Web). Dal punto di vista teorico, le risorse non provenivano da un'unica fonte istituzionale: la definizione delle variabili ambientali e delle loro relazioni è stata ricostruita attraverso la consultazione di materiale tecnico e divulgativo reperito online (documentazione meteorologica, articoli di modellazione climatica, blog scientifici e dataset pubblici di enti quali FAO/FAOSTAT ed Eurostat), integrato dall'utilizzo di sistemi di intelligenza artificiale generativa, sfruttati per ottenere una sintesi affidabile dei range tipici, delle dipendenze fisiche più comuni e delle pratiche standard con cui parametri come temperatura, umidità relativa, precipitazioni, umidità del suolo e vento vengono descritti e correlati. L'IA ha aiutato quindi a definire entità e metriche coerenti con le pratiche comunemente adottate nella rappresentazione dei fenomeni climatici, assicurando che la simulazione riflettesse relazioni fisiche attendibili e strutture dati compatibili con le analisi previste dal progetto: la simulazione dei dati, infatti, vede una baseline deterministica con variabilità controllata, per produrre trend credibili per ciascuna entità del dominio. Inizialmente garantire la coerenza tra dati simulati ambientali e dati produttivi si è rivelato un'importante sfida di progetto e, per affrontare questa esigenza, il sistema è stato progettato in modo tale che ad ogni tick temporale (ovvero ad ogni intervallo di calcolo), venga eseguito un aggiornamento schedato unico: in primis si calcolano i dati ambientali, e successivamente, nello stesso ciclo, si generano i dati produttivi basati su quelli appena calcolati. Questa scelta assicura che le variabili produttive riflettano immediatamente gli effetti delle condizioni ambientali correnti, mantenendo un legame logico costante. Per evitare che le simulazioni producano valori fisicamente o logicamente impossibili, sono stati applicati dei clamp sui range e sui delta di variazione. Per "clamp" intendiamo un'operazione che limita un valore entro dei limiti prestabiliti: se il valore calcolato è inferiore al minimo, viene portato al minimo; se è superiore al massimo, viene ridotto al massimo consentito. In questo contesto, i clamp servono sia per evitare salti troppo grandi tra un tick e il successivo (controllando il

delta), sia per restare entro ambiti realistici (il range). Inoltre, nel sistema è prevista una correzione preventiva: i valori fuori scala, ossia quelli che eccedono i limiti consentiti anche dopo l'applicazione dei clamp, vengono aggiustati prima di essere trasmessi alla dashboard o salvati nella memoria dati. Questa strategia garantisce che la simulazione rimanga stabile e realistica, evitando l'iniezione di dati che potrebbero invalidare l'analisi o rendere la visualizzazione dei grafici poco credibile.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

Il primo obiettivo: “Integrare e utilizzare varie fonti di dati per alimentare un dashboard interattivo”, viene proposto con le seguenti forme: il DataSimulatorService provvede alla generazione di dati ambientali e produttivi attraverso meccanismi di randomizzazione controllata, mentre i modelli di dominio (EnvironmentalData, ProductionData, Crop, Region) definiscono in modo strutturato le entità di ingresso. I repository (EnvironmentalDataRepository e ProductionDataRepository) assicurano la persistenza dei dati simulati. È inoltre presente una componente real-time supportata da WebSocketConfig e dal relativo controller, che regola il flusso comunicativo tra backend ed interfaccia utente. Il lavoro di aggregazione e sintesi dei dati nei grafici soddisfa l'ulteriore requisito: “sintetizzare le informazioni in un formato immediatamente utile e interpretabile dai decisori aziendali”, garantendo una rappresentazione visiva chiara ed operativa per l'utente finale.

Il secondo obiettivo: “Implementazione del frontend tramite Spring Boot e Thymeleaf”, è stato anch'esso raggiunto come evidenziato dalla presenza dei template dashboard.html e simulate.html nella directory src/main/resources/templates/ gestiti dai controller Spring. Il requisito relativo a: “interazioni dinamiche e personalizzate in base alle esigenze specifiche degli utenti” è stato promosso implementando sulla pagina descritta da simulate.html, un'elevata configurabilità dei parametri di generazione. È stata apportata una logica simile anche su dashboard.html, che offre a questo proposito filtri dedicati per area geografica, tipologia di metrica ed intervallo temporale.

Il terzo obiettivo: “Applicare principi di visualizzazione dei dati e di user experience”, è stato perseguito mediante l'inserimento di elementi informativi volti a facilitare l'interazione con il sistema: simulate.html integra una sezione esplicativa (“How it works”), mentre dashboard.html include suggerimenti operativi (“Tips”) ed una descrizione volta a chiarire le differenze tra “Live mode” e “Range mode”. L'interfaccia, progettata in ottica responsive, adotta una gerarchia informativa chiara, evita il sovraccarico cognitivo e mantiene coerenza visiva e funzionale attraverso l'uso uniforme di colori, tipografie e componenti interattivi in entrambe le pagine, garantendo così una navigazione lineare e prevedibile. Infine, per segnalare all'utente l'arrivo di nuovi dati durante la “Live Mode”, è stato introdotto un messaggio informativo “toast”, posizionato in un'area non critica della dashboard, così da non sovrapporsi alle componenti rilevanti dell'interfaccia.

È inoltre possibile produrre l'artefatto eseguibile in target/PW_1_10-1.0.0.jar tramite il comando standard “mvn clean install”. Per altri dettagli più esaurienti, si rimanda all'ancora sul [README.md](#) che fornisce una documentazione completa per la build e run dell'applicazione.

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

L'applicazione sviluppata si colloca all'interno del più ampio ambito dei sistemi di monitoraggio, modellazione e supporto decisionale per il settore primario, un'area che negli ultimi anni ha registrato una crescente attenzione in virtù dei processi di digitalizzazione agricola, della disponibilità di dati ambientali e produttivi e della necessità di affrontare con strumenti quantitativi le sfide legate alla sostenibilità. La scelta del nome dell'elaborato, “RealTime Decision Support System per l'AgriTech”, vuole dare un ruolo centrale ai sistemi DSS, ovvero quelli che supportano il

processo decisionale umano, non prendendo decisioni in autonomia ma fornendo dati, modelli e analisi per aiutare chi deve decidere nella valutazione di scenari complessi.

Per la prossima sezione, si è immaginato un contesto produttivo che vede un'azienda fittizia chiamata GreenData con la necessità di gestire dati provenienti da sensori e/o sistemi IoT impiegati nel campo Agritech.

Descrizione dei principali aspetti progettuali

(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Contesto e processo produttivo.

GreenData è un'impresa operante nel settore agricolo primario, con unità produttive distribuite su diverse regioni italiane. Le superfici coltivate si estendono dalle aree continentali del Nord, come Lombardia, Piemonte, Emilia-Romagna e Veneto, fino ai territori a clima mediterraneo del Centro-Sud, comprendendo Toscana, Lazio, Campania e Sicilia. L'azienda concentra la propria attività sulla produzione di colture estensive quali frumento, mais, riso, soia e girasole.

La strategia gestionale di GreenData integra pratiche agronomiche tradizionali con tecniche di agricoltura di precisione, mirando a incrementare la resa unitaria, ottimizzare l'uso di risorse idriche e input chimici, aumentare la resilienza nei confronti delle variazioni climatiche e dei cicli di mercato, e generare dati operativi a supporto delle decisioni relative a semina, irrigazione, trattamenti e raccolta.

Il processo produttivo dell'azienda si articola in diverse fasi. La pianificazione colturale prevede l'individuazione delle colture per ciascuna area sulla base delle rotazioni colturali, delle caratteristiche pedologiche e delle previsioni di mercato, definendo le superfici destinate a ogni coltura e gli obiettivi produttivi in termini di resa per ettaro. La preparazione del terreno e la semina includono l'esecuzione delle lavorazioni del suolo, la predisposizione dei letti di semina, la distribuzione dei semi e la rilevazione dei parametri iniziali come umidità e temperatura del suolo. Durante la fase di monitoraggio colturale, vengono acquisiti periodicamente dati ambientali quali temperatura, umidità relativa, precipitazioni, umidità del suolo e velocità del vento, permettendo l'attuazione di interventi mirati come irrigazione, concimazione e trattamenti fitosanitari basati su soglie operative. Le attività di raccolta e post-raccolta comprendono la registrazione delle quantità effettivamente prodotte, la gestione logistica e la rilevazione dei costi, mentre l'analisi e il reporting aggregano le informazioni in indicatori visualizzati tramite la dashboard interattiva "Farm Operation Dashboard" che fornisce supporto alle decisioni aziendali.

[Tabelle di dati random generati]

GreenData ha sviluppato un sistema di acquisizione dati che comprende sia variabili ambientali sia dati di produzione. I dati ambientali includono un'indicazione temporale, regione, temperatura in gradi Celsius, umidità relativa in percentuale, precipitazioni in millimetri, umidità del terreno in percentuale e velocità del vento in km/h. Questi dati vengono utilizzati per monitorare le condizioni agro-meteorologiche, valutare lo stress idrico delle colture e attivare interventi automatici e/o sensibilizzare interventi manuali. I dati di produzione includono un'indicazione temporale, regione, coltura, quantità raccolta in kg, giorni di crescita, resa per ettaro, costo di produzione in euro, prezzo di mercato e ricavo. Queste informazioni consentono di effettuare analisi economico-produttive, confrontare le performance tra aree e colture e generare report aziendali dettagliati.

Un'idea rapida e pratica si può evincere dall'apposita ancora sul [README.md](#), per visionare i dataset completi si raccomanda l'utilizzo dell'applicazione: in seguito ad una generazione, sarà possibile ispezionare i dati tramite la console dell'in-memory db H2 (<http://localhost:8080/h2-console>). Nella pagina di login si dovrà procedere all'autenticazione con username: sa e password: password, lasciate volutamente di default per semplificazione. In seguito, nello spazio apposito sulla destra, si potrà interrogare il db con semplici query sql: Es. "SELECT * FROM

ENVIRONMENTAL_DATA“ restituirà la tabella completa di dati ambientali, allo stesso modo “SELECT * FROM PRODUCTION_DATA” restituirà la tabella completa con i dati di produzione.

[Codice Java per la dashboard]

Possiamo adesso esplodere il progetto pubblicato sul repository pubblico: https://github.com/AriannaSocci/PW_1_10.git.

L'ingresso principale dell'applicazione, è rappresentato dalla classe DataGeneratorApplication.java, che avvia l'intero sistema. Parallelamente, WebSocketConfig.java si occupa della configurazione della comunicazione in tempo reale, utilizzando STOMP su WebSocket con fallback SockJS: grazie a questa configurazione, il server può gestire connessioni persistenti e inviare aggiornamenti in modo efficiente secondo il paradigma publish-subscribe.

Questa architettura è supportata da Spring, che consente di definire endpoint STOMP e broker semplici, come descritto nella documentazione ufficiale.

La logica dei controller è incapsulata in DashboardController.java, il quale espone gli endpoint MVC necessari per la simulazione e per la visualizzazione della dashboard tramite Thymeleaf e Javascript. Sul fronte dei servizi, ho definito diverse classi chiave: DataSimulatorService.java genera i dati ambientali e produttivi, DashboardService.java si occupa dell'aggregazione e della preparazione delle strutture dati da passare ai template, mentre WebSocketDataService.java ha il compito di inviare in tempo reale gli aggiornamenti tramite WebSocket agli utenti sottoscritti.

Per quanto riguarda la persistenza, i dati vengono memorizzati in due repository distinti: EnvironmentalDataRepository.java per i dati ambientali e ProductionDataRepository.java per i dati di produzione. I modelli su cui si basa la simulazione sono definiti dalle classi EnvironmentalData, ProductionData, Crop e Region, che rappresentano le entità del dominio.

[Resoconto del processo di sviluppo del codice]

In seguito all'elaborazione ed all'analisi dei requisiti che hanno permesso di definire le variabili ambientali e produttive da includere nella simulazione, è stato progettato e realizzato il disegno architetturale, con la suddivisione del progetto secondo il pattern MVC, la scelta del database H2 e l'adozione di WebSocket per consentire aggiornamenti in tempo reale.

Come già descritto, la parte iniziale di sviluppo vero e proprio ha riguardato la base dati utilizzata per le simulazioni: la generazione effettiva dei dati avviene una volta che la dashboard dedicata (simulate.html) è stata compilata. Questo template esegue un ciclo tra un tempo di inizio e uno di fine, creando campioni sia ambientali sia produttivi per ogni regione e coltura; tali dati vengono poi salvati nei relativi repository, garantendo una struttura persistente e coerente su cui il sistema può basarsi per la visualizzazione e l'elaborazione. Maggiori dettagli riguardo gli algoritmi di simulazione dei dati sono presenti su questa ancora del [README.md](#) ma, generalmente, possiamo dire che la baseline deterministica rappresenta il valore centrale atteso di ogni variabile in condizioni standard. Vengono successivamente applicate perturbazioni casuali limitate con l'obiettivo di introdurre oscillazioni naturali senza generare valori irrealistici o instabili. Le variabili ambientali incorporano rumore di tipo AR(1)-like, consentendo fluttuazioni autocorrelate nel tempo. La stagionalità è modellata tramite modulazioni semplici, implementate tramite un costrutto analogo a SeasonInfo, che permette di simulare ciclicità annuali o periodiche.

Successivamente è stato implementato anche lo sviluppo delle dashboard: particolare attenzione è stata rivolta alla gestione strutturale e funzionale del canale WebSocket. Il parsing dei payload ha richiesto un'attenzione specifica, prevedendo la validazione dei messaggi STOMP in ingresso e la gestione di valori anomali o assenti, così da preservare l'integrità dei dati visualizzati. Successivamente, il rendering è stato ottimizzato attraverso l'aggiornamento incrementale dei grafici, sfruttando le funzionalità offerte da librerie come Chart.js che consentono di aggiungere nuovi punti senza ricostruire il grafico a ogni tick, garantendo così fluidità anche in presenza di stream dati

continui. Dal punto di vista architetturale, è stata mantenuta una netta separazione tra la logica di presentazione, demandata a Thymeleaf, e la logica dinamica implementata in JavaScript: il primo ha fornito lo scheletro statico dell'interfaccia, mentre il secondo ha gestito la reattività e gli aggiornamenti in tempo reale, favorendo un design più pulito e facilmente verificabile.

Sono stati inoltre considerati aspetti di accessibilità e responsività, adottando layout flessibili per assicurare la corretta fruizione della dashboard anche su schermi di dimensioni ridotte. Infine, la sincronizzazione tra più grafici ha rappresentato un ulteriore elemento progettuale rilevante: garantire che variabili come temperatura, umidità o velocità del vento venissero aggiornate nello stesso tick ha permesso di preservare la coerenza temporale e rendere visibili eventuali correlazioni tra fenomeni, migliorando l'interpretabilità complessiva della visualizzazione.

Una volta soddisfatti i requisiti e completato lo sviluppo, l'applicazione è stata testata manualmente: dalla generazione iniziale, prestando particolare attenzione alla coerenza dei dati, all'interfaccia utente ThymeLeaf, assicurando che gli aggiornamenti live dal backend fossero correttamente aggregati e visualizzati.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

L'applicazione può essere considerata un prototipo di Decision Support System (DSS), progettato per monitorare e analizzare l'evoluzione temporale di indicatori agricoli provenienti da sensori e sistemi IoT, con aggiornamenti in tempo reale. La generazione simulata dei dati si ispira a dinamiche plausibili, consentendo di verificare le logiche di elaborazione prima di un'eventuale integrazione con dataset istituzionali o dati di campo reali.

Gli ambiti di applicazione di questo elaborato risultano molteplici: in ambito di ricerca e sperimentazione, la piattaforma consente di testare modelli predittivi e algoritmi di analisi dei dati agricoli senza la necessità di disporre di dati reali iniziali, permettendo di valutare l'efficacia delle metodologie implementate; dal punto di vista didattico e dimostrativo, l'applicazione offre la possibilità di illustrare le potenzialità dei DSS, favorendo la formazione di personale tecnico e l'informazione di decisori. Inoltre, la piattaforma rappresenta un ambiente utile per la validazione di infrastrutture software, consentendo di verificare scalabilità, affidabilità e prestazioni prima dell'integrazione con sistemi reali. Una volta collegata a dati effettivi, il sistema può supportare le decisioni operative, ottimizzando interventi agricoli, gestione delle risorse e pianificazione strategica.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti):

L'architettura modulare ed orientata ai servizi dell'applicazione consente l'integrazione futura con dataset reali, permettendo di sostituire progressivamente i dati simulati con flussi certificati. La dashboard potrebbe essere ulteriormente arricchita con funzionalità decisionali avanzate, quali sistemi di allerta, analisi comparative tra regioni o scenari "what-if". Inoltre, l'infrastruttura basata su WebSocket favorisce la scalabilità, ampliando l'utilizzabilità del sistema in contesti sia professionali sia didattici. Per quanto concerne la portabilità del sistema, essendo ospitato su GitHub, un futuro sviluppo facilmente perseguibile riguarda l'integrazione di una CI (Continuous Integration) con le Github Actions per una build con Docker. Nello specifico, si tratta di scrivere dei file.yaml che verranno eseguiti sul trigger di un determinato evento da GitHub stesso, questi potrebbero eseguire la build del codice con un Dockerfile che permette la sua containerizzazione e una successiva push su GHCR (GitHub Container Registry) che si presta ad uso gratuito fintanto che le immagini restano pubbliche. Questo sviluppo permetterebbe agli utenti di non dover installare tutte le dipendenze richieste dal progetto e di non subire i vincoli imposti dal proprio OS (Sistema Operativo), ma renderebbe sufficiente alla sua esecuzione la sola installazione di Docker.

Una volta presente l'infrastruttura che ospiterà l'applicazione, per mezzo delle GitHub Actions sarà possibile implementare anche un job per la CD (Continuous Delivery) che potrebbe essere eseguito in maniera automatica una volta che la build e la push del codice siano state correttamente eseguite sul Docker Registry. Questa action permetterà il deploy del sistema sull'architettura fisica di destinazione rispettando, se necessario, la necessità di ridondanza per avere un'alta affidabilità (HA – High Availability).

Tuttavia, attualmente, un punto d'attenzione e miglioramento riguarda i dati utilizzati, che sono interamente simulati e pertanto non idonei ad analisi statistiche o valutazioni operative reali. Inoltre, le tipologie di raccolti potrebbero essere più vari; infatti, con una simulazione sull'ultima settimana in inverno, si riscontreranno dati di produzione correlati soltanto al grano, rendendo il grafico di poco utile ed esplicativo. Un altro punto d'attenzione concerne il motore di simulazione che impiega modelli piuttosto semplificati, che non riproducono completamente la complessità dei processi agricoli né la piena variabilità climatica; in futuro, tali modelli potrebbero essere integrati con approcci basati su machine learning. Un ulteriore limite è stato individuato sulla dashboard, attualmente orientata alla visualizzazione esplorativa e priva di strumenti avanzati per l'analisi di grandi volumi di dati o per la loro gestione operativa, consideriamo anche che l'organizzazione delle sottoscrizioni STOMP non è stata impostata secondo un approccio modulare, definendo topic distinti al fine di separare le diverse tipologie di dati e rendere l'architettura facilmente estendibile e manutenibile. Infine, l'utilizzo di un database in-memory e l'assenza di persistenza a lungo termine riducono l'applicabilità del sistema in scenari produttivi o multi-sessione. Sulla base di questo ultimo punto si è pensato ad un'altra implementazione futura: per garantire la portabilità ed il versionamento, si potrebbe quindi implementare una containerizzazione integrando ad esempio strumenti OSS (Open Source System) come LiquiBase.